

Kimi K3 技术分享

开源前沿智能 · 2.8T MoE / 1M 上下文

重点：结构设计背后的"为什么" · 预训练 · 预训练 Infra

(模型结构基础已由前序分享覆盖，本场只讲动机与原理；强化学习简化)

资料：K3 技术报告 · Attention-Residuals · FlashKDA · MoonEP · Kimi Linear

本场路线图（约 1 小时）

1. 定位与总纲（5 min）

- 为什么做 3T · 三轴信息流

2. 结构 · 为什么这么设计（20 min）

- KDA / Gated MLA·NoPE
- Attention Residuals（深度维）
- Stable LatentMoE（SiTU-GLU / QB）
- Per-Head Muon · 与 DeepSeek/Qwen 对比

3. 预训练（10 min）

- Scaling Law · 长上下文 · QAT

4. 预训练 Infra（重点，18 min）

- KDA 协同（FlashKDA / KCP）
- MoonEP 完美负载均衡
- 显存高效训练

5. RL（简化）+ 收尾（7 min）

🎯 三个“钩子”：KDA 管跨 token，AttnRes 管跨 layer，LatentMoE 管跨 channel。

Part 0 · 定位与总纲

为什么是 3T？三轴信息流

为什么把底座推到 3T 级？

- LLM scaling 有 **两条轴**：
 - **轴一 · 预训练规模**：更大模型 + 更多数据
 - **轴二 · test-time compute**：RL + 推理 effort + 长程交互
- 现状：开源在 **轴二** 猛进（推理 / agentic RL），但 **轴一停在 1T 级** 附近
- 风险：都在相似规模底座上堆 RL → 开源 **互相收敛**，与闭源前沿 **差距扩大**
- **K3 的选择：两轴同时推到前沿** —— 底座到 3T 级，同时把 RL / effort / 交互推到 **1M 上下文**

结果：2.8T 总参 / 104B 激活 / 1M 上下文，整体 scaling 效率相比 K2 $\approx 2.5\times$

总纲：沿三个维度"扩展信息流"

所有结构创新都在回答同一个问题 —— 信息如何更高效地流动？

维度	机制	解决什么
序列维 (token mixing)	Hybrid Attention : 3×KDA + 1×Gated MLA	长序列高效混合 + 保留全局高容量注意力
深度维 (layer mixing)	Attention Residuals	每层可选择性检索所有前层表示
宽度维 (channel mixing)	Stable LatentMoE : 896 选 16	稀疏扩展通道混合容量
训练稳定	Per-Head Muon + QB + 有界化	在 2.8T/896 专家/MXFP4 下稳住优化

记住这张表 —— 后面每一节都挂在这三个维度上。

K2 → K3 规格对照

项	Kimi K2	Kimi K3	Δ
总参 / 激活	1.04T / 32.6B	2.78T / 104B	+167% / +220%
层数	61	93	+52%
注意力	全 MLA	69 KDA + 24 Gated MLA (3:1)	混合
路由专家 / 激活	384 / 8	896 / 16 (+2 shared)	+133% / +100%
Latent MoE 维	—	3584 (0.5×hidden)	新
激活函数	SwiGLU	SiTU-GLU	新
训练上下文	128K	1M	8×
位置编码	RoPE	NoPE	新
优化器	Muon	Per-Head Muon	精化
量化	—	MXFP4 权重 / MXFP8 激活 (QAT)	新

Part 1 · 模型结构

重点：为什么这么设计（基础从简）

序列维：Hybrid Attention = 3×KDA + 1×MLA

- 每个 block：3 层 KDA（线性注意力）+ 1 层 Gated MLA（全局注意力），全网重复；末尾补 1 层 MLA
→ 最后一层一定是全局注意力
- 为什么混合？
 - KDA 承担 绝大多数 token mixing：固定大小递归状态，长序列便宜
 - 线性注意力的 软肋 = 长程精确召回 → 用 1/4 的全局 MLA 兜底
 - 3:1 = "效率 vs 召回" 的工程折中（继承自 Kimi Linear）
- 好处直接体现在 1M 上下文：KV cache 大幅下降、解码吞吐大幅提升

DeepSeek-V3 = 全 MLA；Qwen3 = GQA。K3 用"线性为主 + 全局兜底"换长上下文效率。

KDA 是什么：带通道级遗忘门的 delta rule

单头递归（固定大小状态 $S_t \in \mathbb{R}^{d_k \times d_v}$ ）：

$$S_t = (I - \beta_t k_t k_t^\top) \text{Diag}(\alpha_t) S_{t-1} + \beta_t k_t v_t^\top, \quad \tilde{o}_t = S_t^\top q_t$$

- $\text{Diag}(\alpha_t)$ ：**通道级**（每个特征维独立）保留因子 $\rightarrow$ 比 Gated DeltaNet / Mamba-2 的 **头级标量门** 更细粒度
- $(I - \beta_t k_t k_t^\top)$ ：delta rule 的**先擦除旧关联，再写入新的 $k_t v_t^\top$** ， β_t 控写入强度
- 参数化：q/k/v 走 **ShortConv + Swish**，q/k 再 **L2Norm**；衰减 logit 低秩投影
- **一句话**：用固定大小 RNN 记忆，做到“可控遗忘 + 精确改写”

KDA 演进 ①：给衰减门"加下界" → 系统大收益

- **问题**：chunkwise 里要用累计衰减的**倒数** $1/\Gamma$ 重标定 key； Γ 是一堆 (0,1) 连乘 → 倒数**无界**、BF16 下溢出
- Kimi Linear 的对角 tile 只能用 **逐位置对 (position-pair)** 计算 → **上不了 Tensor Core**，成 intra-chunk 瓶颈
- **K3 改法**：把 log-decay 从"无界负 Softplus"换成 **带下界的缩放 sigmoid**

$$g_t = g_{\min} \text{Sigmoid}(e^{A_h} z_t) \in (g_{\min}, 0), \quad \alpha_t = e^{g_t}, \quad g_{\min} = -5$$

- **数学后果**： $\alpha_t > e^{-5} \rightarrow$ 16-token tile 累计 log-decay $\in (-80, 0) \rightarrow$ 重标定因子 $< e^{80}$ **落在 BF16 范围**
- **系统收益**：对角块也能用 **稠密 Tensor Core GEMM**，彻底删掉 position-pair 路径

💡 典型的"算法-系统协同"：一个纯数学的下界，换来 kernel 的硬件友好。

KDA 演进 ②：全秩输出门

Kimi Linear 用**低秩**输出门 → K3 换成 **输入相关的全秩**投影：

$$y_t = W_o \left[\text{Sigmoid}(W_g x_t) \odot \text{RMSNorm}(\tilde{o}_t) \right]$$

- **为什么**：让每个 token 更充分地**调制**从递归记忆读出的通道 → 表达力更强
- 与 K3 的 Gated MLA 输出门**统一参数化**（一致性 → 工程/训练更省心）

Gated MLA + NoPE : 全局层为什么不加位置编码

- **MLA** (源自 DeepSeek-V2) : 把每 token 的 KV 压成低维 latent $c_t = W_c x_t$ 缓存, 用时上投影重建 $\rightarrow$ KV cache 大幅缩小, 保留全局注意力
- **改动 1 · NoPE** : 所有 MLA 层 **不加任何显式位置编码**
 - 位置信息由中间的 **KDA 层通过递归门控/衰减隐式提供**; MLA 只做"无约束全局内容交互"
 - 副产品: **扩上下文零改动** —— 不用重调 RoPE base、不用 YaRN 插值 $\rightarrow$ 直接外推 1M
- **改动 2 · 全秩输出门** : 与 KDA 一致
- 数值工程: 训练时注意力输出保 **FP32** 纠正 flash-attn 有偏 rounding, 并重设计 kernel 用其与 KV staging 重叠

深度维：为什么需要 Attention Residuals

洞见：标准残差 = 深度维的 RNN。

展开 PreNorm : $h_l = h_1 + \sum_{i < l} f_i(h_i)$ —— 每层用固定单位权重把所有前层加起来

- 三个后果：

- i. 无选择性访问：attention 层和 MLP 层拿到同一个聚合态

- ii. 不可逆损失：早层信息被埋，深层无法选择性找回（实测可裁掉不少层）

- iii. 输出膨胀：PreNorm 下 hidden 幅度随深度 $O(L)$ 增长，稀释每层贡献、破坏稳定

类比：序列维当年用 attention 取代 RNN 递归 —— 深度维也该如此。

Attention Residuals : 深度维 softmax 注意力

每层 l 用一个 **可学伪 query** w_l , 对"embedding + 所有前层输出"做 softmax 注意力 :

$$\phi(q, k) = \exp(q^\top \text{RMSNorm}(k)), \quad \alpha_{i \rightarrow l} = \frac{\phi(w_l, k_i)}{\sum_j \phi(w_l, k_j)}, \quad h_l = \sum_{i=0}^{l-1} \alpha_{i \rightarrow l} v_i$$

- $k_i = v_i = \{ \text{第 0 位是 embedding } h_1 ; \text{其余是层输出 } f_i(h_i) \}$
- **RMSNorm(k)** : 防大幅度层主导权重
- **理论视角** : 标准残差 = 深度维**线性**注意力 ; AttnRes = 深度维 **softmax** 注意力
→ 完成序列维当年"linear → softmax"的同款跃迁
- 深度 $L < 100 \rightarrow O(L^2 d)$ 算力可忽略 , 代价是 $O(Ld)$ 显存/通信

Block AttnRes : 把开销从 $O(Ld)$ 降到 $O(Nd)$

- 把 L 层切成 N 个 block : 块内标准残差求和得 b_n , 块间才做全注意力 (只对 N 个块表示) ; $b_0 = h_1$
- $N=1$ 退化为标准残差 , $N=L$ 恢复 Full ; 实测 $N \approx 8$ 拿回大部分收益
- **K3** : 93 层 $\rightarrow$ 8 个 block (每块 12 层) + embedding , 共 9 个源
- **收益 (论文级)** :
 - 同算力匹配 **多 1.25 $\times$ 算力** 的 baseline
 - hidden 幅度随深度**有界**、梯度层间**更均匀** (缓解 PreNorm dilution)
 - 下游全面提升 : GPQA-Diamond **+7.5**、HumanEval **+3.1**
- **推理友好** : w_l 与前向解耦 $\rightarrow$ 块间可并行 ; 块内用 **online softmax** 合并 , 与 Full 严格等价 ; 推理延迟开销 **<2%**

宽度维：LatentMoE 让"896 选 16"可负担

常规 MoE：每个被选专家吃**完整 d 维** token → 通信/权重流量随激活多重度爆炸

LatentMoE：分离"模型宽度"与"路由专家宽度"

$$z = W_{\downarrow} x \in \mathbb{R}^{\ell}, \quad u = \sum_{i \in \text{Top}k} p_i E_i^{\text{routed}}(z), \quad y = \sum_j E_j^{\text{shared}}(x) + W_{\uparrow} \text{RMSNorm}(u)$$

- **shared 专家走全宽**（共性变换）；**routed 专家在压缩 latent 空间** $\ell = 3584 = 0.5 \times \text{hidden}$ 里算
- K3：shared=2，routed=**896 选 16**，稀疏度 ≈ 56 （组织沿用 DeepSeekMoE）

只有把 routed 专家压到 latent 空间，896 选 16 的通信/显存才付得起。

极端稀疏放大两个失效模式 → 三招治

896 专家把 vanilla 设计的两个问题放大：

- 1. routed 分支近似 4 连乘矩阵 ($W_{\downarrow} \cdot \text{gated-FFN} \cdot W_{\uparrow}$) → 病态、内部激活爆炸
- 2. 近 10^3 专家的负载均衡超出现有 aux-loss-free bias 更新能稳住的范围

Stable LatentMoE 的三招：

招	治什么	做法
Normalized LatentMoE	尺度敏感	u 与 $W_{\uparrow}$ 之间插 RMSNorm ；顺带改善验证 loss
SiTU-GLU	激活爆炸	对 Swish 线性因子 + up 分支各套 softcap
Quantile Balancing	大规模均衡	bias 设为 margin 分布的分位数

SiTU-GLU : 有界又保留 SwiGLU 的形状

- **背景** : SwiGLU 两个乘子 ($x\sigma(x)$ 和 x) **都无界** $\rightarrow$ 大坐标产生激活离群点、低精度易溢出
- **SiTU-GLU** : 对 Swish 线性因子和 up 分支各套平滑 cap $\beta \tanh(\cdot/\beta)$

$$\text{SiTU-GLU}(x) = \beta_1 \tanh\left(\frac{W_g x}{\beta_1}\right) \odot \text{Sigmoid}(W_g x) \odot \beta_2 \tanh\left(\frac{W_u x}{\beta_2}\right)$$

- 超参 $\beta_1=4$ (门)、 $\beta_2=25$ (up)
- 三条数学性质：
 - i. 近原点 $\beta \tanh(z/\beta) = z + O(z^3/\beta^2) \rightarrow$ **一阶匹配 SwiGLU**
 - ii. $\beta_1, \beta_2 \rightarrow \infty$ 逐点恢复 SwiGLU
 - iii. **输出有界** $\|\cdot\|_\infty \leq \beta_1 \beta_2 = 100$
- vs 硬 clamp : 平滑 cap 在非饱和区**保留非零梯度** $\rightarrow$ 训练更好 ; 且利于 MXFP4/8 QAT

Quantile Balancing : 从"符号步长"到"分位数"

- **背景** : aux-loss-free 路由给 router 分数加**专家偏置** b_j 影响 Top-k , 但 b_j **不进** softmax 权重 (不改混合、不干扰 router 梯度)
- DeepSeek-V3 : **固定步长符号** $b_j \leftarrow b_j + \gamma \text{sign}(\text{负载误差}) \rightarrow \gamma$ 大震荡 / 小收敛慢 ; 896 专家更难稳
- **QB 洞见** : 把均衡看作**最优平衡指派** (每专家恰收 $q = mk/n$) $\rightarrow$ 线性松弛 + 对偶 $\rightarrow$ 专家侧对偶变量的闭式解 = **margin 分布的 $(1 - k/n)$ 分位数**

$$\hat{b}_j^{(t+1)} = -\text{quantile}_{1-k/n}(s_{:,j} - \alpha^{(t)}), \quad b^{(t+1)} = \hat{b} - \text{mean}(\hat{b})$$

- 与符号法的关系 : 该对偶目标的次梯度 = "目标负载 - 实际负载" ; SignSGD 一步就退化成 DeepSeek 的符号更新
- **QB 直接跳到精确坐标最小值** $\rightarrow$ 无 lr 类超参、几步即均衡

QB 工程落地：直方图全局估计

- 全局 batch 的 margin 有**数百万**、分散在各 rank/累积步 → 精确分位数不可行
- 每专家维护一个**直方图**（对 $r_{i,j} = \alpha_i - s_{i,j}$ 分箱）
- 一次 **all-reduce 求和 bin 计数** → 从池化计数恢复分位数
- counts 可加 → **与 token 如何分片无关** → 得到的是**全局** batch 的分位数
- 通信量仅每层每步 $n \times B$ 个整数（ $B \approx 1000$ ，误差 $\sim \text{几} \times 10^{-3}$ ）；可再对分位数做 EMA 降噪

负载均衡从"启发式步长"升级为"有理论最优解 + 可扩展估计"。

Native Vision + Per-Head Muon (速览)

原生多模态

- 文本/图/视频 **同一 backbone、同一 context、单一 next-token 目标**
- **MoonViT-V2 (401M)** 从零训练，不再用 SigLIP 初始化
 - 理由是**稳定性**：SigLIP-init 梯度范数持续偏高+尖峰
 - 性能打平 → 大规模 MLLM **不必**依赖对比预训练

Per-Head Muon

- Muon = 动量 + Newton–Schulz 正交化
- **改动**：Q/K/V 投影 **按 head 切块、逐 head 正交化**
- **为什么**：整矩阵正交化下，大尺度 head 主导共享更新方向 → 逐 head **均衡各 head 更新尺度**
- 顺带：高瘦块 Newton–Schulz 更便宜

与 DeepSeek-V3 / Qwen3 的对比

维度	DeepSeek-V3	Qwen3-MoE	Kimi K3
注意力	全 MLA	GQA + QK-Norm	3×KDA + 1×MLA 混合
位置编码	RoPE	RoPE	NoPE (KDA 隐式)
深度维聚合	标准残差	标准残差	Attention Residuals
MoE	256 选 8 (+1)	128 选 8 (无 shared)	896 选 16 (+2)
专家宽度	全宽	全宽	Latent 压缩 0.5×
激活函数	SwiGLU	SwiGLU	SiTU-GLU (有界)
负载均衡	符号步长	global-batch aux-loss	Quantile Balancing
优化器	AdamW	AdamW	Per-Head Muon
量化	FP8 训练	—	MXFP4/8 QAT

对比小结：K3 的取舍与优劣

- 一句话：K3 = 把 DeepSeek 系地基（MLA + shared/routed MoE + aux-loss-free）上，叠 KDA（线性注意力）+ AttnRes（深度维）+ LatentMoE（宽度稀疏）+ Per-Head Muon + QAT，把规模与上下文同时推到 3T / 1M

✓ 优势

- 1M 上下文 KV/算力显著更省（KDA + NoPE）
- 稀疏度 ~56、专家特化空间大 → scaling 效率高
- 低精度更稳、部署成本低（SiTU-GLU + QAT）
- 均衡无超参、几步收敛（QB）

⚠ 代价

- 结构 + 系统复杂度显著上升
- 线性注意力长程召回靠 1/4 MLA 兜底
- 极端稀疏需一整套稳定化 + MoonEP
- → 所以必须配套 FlashKDA / KCP / MoonEP / 显存管理

Qwen3 走保守稳健路线：工程简单，但 scaling/长上下文上限更受限。

Part 2 · 预训练

Scaling Law · 长上下文 · QAT

预训练数据（速览）

- 四大文本域：**Web / Code / Math / Knowledge** + 大规模视觉语料
- 规则启发式 + 分类器质量打分 + 去重；小模型 ablation 定采样率
- **Rephrasing**（沿用 K2）：对 knowledge/math 用风格与视角多样化 prompt 改写、chunk-wise 自回归生成、**对源文档保真校验**
- **视觉**：坐标监督给绝对 + 归一化 [0,1] 双格式（精确 + 分辨率鲁棒）；大规模 **programmatic 多模态数据**——代码片段配其渲染结果（SVG / 3D / 网页 / 游戏 / CAD）

Scaling Law : 2.5× 与一条方法论

- 结构/数据/训练一起改 → 最优训练 regime 也变 → 专门重调 **batch size / lr / TPP / 模型 shape**
- 结论：整体 **scaling 效率 $\approx 2.5\times$** (held-out OOD 验证)
- **cosine decay 一致优于 WSD**，但关键在方法论 📌

⚠️ 两种调度的最优超参差别很大 —— 用同一套超参比较会不公平地偏袒其中一个。

正确做法：各自独立做 **scaling-law 搜索**，在各自最优下再比。结论才是 cosine 最终 loss 更低。

- 对我们的启示：**比较任何训练技巧，都要"各自调到最优再比"。**

训练配方

- **原生多模态**：语言与视觉 **从训练一开始就联合优化**，视觉/文本 token 交织进单一 next-token 目标
- 优化器：**Per-Head Muon** + K2 的 **weight-clipping**
- MoE 均衡：**Quantile Balancing**
- 调度：**cosine lr + 1% 线性 warmup**；**weight decay = 0.1** 全程
- 上下文课程起步：**8K → 64K**（预训练期）

长上下文扩展到 1M

- **位置**：NoPE → **直接外推 1M**，无需任何位置编码改动（不 rescale RoPE、不 YaRN）
- **数据**：长文/长视频含大量低质（近重复 / 二进制块 / 截断 / 无效日志）→ 专门清洗（精确+模糊去重、视频帧感知哈希、结构校验）
 - 天然长文稀缺 → **上采样**
 - **长度 ≠ 长程能力** → 合成"必须跨全 1M 才能解"的任务（排列拼接多模态文档/子任务），把注意力**训练**在**目标尺度**，防退化成局部模式
- **渐进式四阶段课程**：
 - 预训练期 **8K → 64K**；cooldown 期 **256K → 1M**
 - 把昂贵的长序列计算**集中在训练预算的一小部分** → 经济且渐进适应

部署感知量化（QAT）

- MoE 专家权重（参数显存大头）→ **MXFP4**，激活 → **MXFP8**
- 非专家部分（注意力投影 / latent MoE 投影 / shared 专家 / router）**保持高精度**
- **从 SFT 起全程 QAT**（覆盖 SFT + RL）→ 模型适应量化精度损失
- **RL 时 rollout 与训练用同一量化方案** → **消除 train-infer 失配**

为什么能训得动 MXFP4：SiTU-GLU 有界 + 各处 RMSNorm + KDA 门下界，都是低精度的稳定器。

Part 3 · 预训练 Infrastructure

本场重点

三大系统挑战同处一模型 + 并行组合

K3 罕见地在同一模型叠了三种系统级挑战：

- 混合 **KDA** 注意力（串行递归 vs GPU 宽并行）
- **3T** 级稀疏多模态 训练/推理
- 百万 token agentic 负载

并行组合：PP（+虚拟阶段 VP）+ **EP** + ZeRO-1 DP + **Pipeline ZeRO-2** 梯度分片 + **CP**（**KCP**）

多模态 3T 预训练的三个核心难题 → 逐一解决：

1. token 负载在 EP rank 间不均 → **MoonEP**
2. 激活/梯度/优化器状态超显存预算 → **统一激活管理 / ZeRO-2 / remote-offload**
3. ViT 可变计算暴露在关键路径 → **bubble filling**

KDA 协同 ① : FlashKDA + intra-device CP

KDA 用固定大小递归状态换掉增长的 KV cache —— 好处是状态小易复用，坏处是**串行更新**与 GPU 宽并行冲突，**每个执行阶段是不同瓶颈**。

- **FlashKDA (训练 & prefill)** : chunkwise "块内并行、块间串行"，朴素实现串行传播时 SM 空转
 - CUTLASS kernel，把**块内计算与跨块状态传播 overlap**；拆成 token-parallel 阶段 + head-parallel 递归，各自调优
 - 配合 SKDA 的"下界化衰减"让对角块也上 Tensor Core；作为 flash-linear-attention 自动派发后端
- **intra-device Context Parallelism (长上下文 prefill)** : 纯 TP 只切 head 不缩短递归 → 超长 prefill 时 SM 空转
 - 洞见：**每段状态转移可独立于入态先算、之后精确合成**
 - SM 级 CP planner 把序列切到单 rank 各 SM 并行 → **完全 intra-device、零跨设备通信**

KDA 协同 ② : KCP 跨设备上下文并行

- 线性注意力跨设备 CP 只需传**固定大小递归状态** (softmax 要传随长度增长的 KV 块)
- 但 vanilla "各 rank 从 $S=0$ 算本地态再求和" 对 KDA **不成立** :
 $S_t = M_t S_{t-1} + \beta_t k_t v_t^\top$, token 相关的转移 M_t 要**作用在入态上**
- **KCP 把每段效果分解成两个本地可算量** :

$$S_{[i+1]}^t = \underbrace{\tilde{S}_{[i+1]}^t}_{\text{本地从 0 生成}} + \underbrace{M_{[i+1]}^{t \leftarrow 1} S_{[i]}^{T_i}}_{\text{累计转移作用于入态}}$$

- 两量都能在拿到前序 rank 状态**之前**用本地 token 算完 , 且 rank 级更新**满足结合律**
- → 用 **前缀扫描 (prefix scan)** 恢复各 rank 入态 , 只需一次**固定大小 all-gather** , **线性算力扩展**

思想迁移 : 把 softmax 的 CP 思路 , 用"转移矩阵分解 + 前缀扫描"迁到线性注意力。

MoonEP：为什么需要"完美负载均衡"

常规 EP（如 DeepEP）的痛点：

- token 负载在 rank 间**不均** → 计算不均**拖慢吞吐**（最热 rank 决定延迟）
- routed 激活是**动态 shape** → 显存**碎片化**、每层要 **host-device 同步**取真实 shape → 流水停顿、高不均时 **OOM**

MoonEP 的目标：让每个 rank 恰好收 $S \times K$ 个 token（S=每 rank token 数、K=top-k）

→ 一旦"完美均衡"，就能连锁拿到：**静态 shape + 零拷贝 + 免 host 同步 + 不 OOM。**

MoonEP : 完美均衡 + E/R 上界证明

核心问题：要多少"冗余专家"才能保证总能均衡？

定理 (附录 E)：每 rank 至多 E/R 个冗余专家就一定存在均衡方案，且此界基本紧。
(E =专家数, R =EP size)

构造性证明思路：

- 反复"用一个过载 rank 去填满一个欠载 rank，直到恰好 $S \times K$ "
- 每次填满**一个** rank 且之后不再变 $\rightarrow$ 至多 $R - 1$ 次；每 rank 至多被**一个** rank 填
- $\rightarrow$ 某 rank 的远程 token 来自**单一** rank 的 $\leq E/R$ 个本地专家 ■

意义：每 rank 预留 E/R 冗余槽 $\rightarrow$ 规划**总有可行解、训练永不中断**

(对比 ECHO/UltraEP：预设冗余数或 per-rank cap，无解时被迫停训、还要手调 cap)

MoonEP : 在线规划 · 零拷贝 · 静态 shape

- **在线规划** : 每步精确 ILP 太贵 $\rightarrow$ 离线 ILP 求参照 , 设计 **GPU 规划 kernel** (近最优、开销可忽略、始终满足 E/R 上界)
 - 前向从当前 router 输出规划冗余专家并**预取** ; 反向把其梯度 stage 到本地 reduce buffer , 算完 reduce 回 home rank
- **零拷贝通信** : 融合 permute/unpermute —— token **直接发到远程 rank 的专家分组位置** , 通信 buffer 的**视图直接交给计算**
 - 完美均衡下 buffer 只需固定 $S \times K$ (DeepEP 最坏要 $S \times K \times R$)
- **静态 shape + sync-free** : 完美均衡 $\rightarrow$ MoE 计算 shape **静态已知** $\rightarrow$ **消除每层 MoE host 同步**、降 kernel 启动开销
- **Expert-GEMM 调度** : rank 内每专家 token 数仍偏斜 $\rightarrow$ **workload-aware 调度器** (分析型代价模型 + 离线 autotune) ; shared 专家 GEMM 派单独 stream overlap

显存高效训练 ①

- **统一激活管理器**：每个为反向保存的 tensor 绑一个**可插拔存储后端**
 - 重计算 / 量化 / offload / remote-offload 都只是"存储策略"，可在 tensor 粒度自由组合，用注解声明、与模型代码解耦
 - 单一内存池（避免多流碎片）；激活按层预取 + 与计算 overlap
 - K3 多数激活用 **block-wise FP8 + offload**，逐元素算子配重计算
- **Memory-efficient MoE**（借鉴 SonicMoE）：把 permuted probs 梯度**数学变换**成只依赖中间激活 + 上游梯度的形式，消除反向对前向 `output` 的依赖；group GEMM 反向**重算 dispatch** 恢复输入（通信可与反向 overlap）
- **Memory-efficient AttnRes**：block 表示在边界层生成一次、后续层共享、常驻 GPU；AttnRes 全部 checkpointing 包裹 → **每层保存的激活与标准残差相同**；PP 用 cache-based 通信只增量传新 block

显存高效训练 ②

- **跨 PP rank 均衡激活**：interleaved 1F1B 下激活分布不均（越靠后 rank 常驻越少）→ 用 **Mooncake Transfer Engine** 把激活 remote-offload 到其他 PP rank 内存 → 均衡、防 OOM
- **Pipeline ZeRO-2 梯度分片 + offload**：梯度按 DP rank 分片，分片梯度存 **CPU 内存**降 GPU 峰值，double grad buffer 留 GPU
- **P2P-based Muon 正交化**：
 - Newton-Schulz 需**完整参数矩阵**，但分布式优化器把参数分片了 → 需先 gather
 - 朴素全参 all-gather = 显存 + 通信双瓶颈
 - **K3：每 rank 只 P2P 取回自己负责参数的分片**（向 owner rank 点对点）→ 消掉全参 buffer、降显存与通信；按 model-chunk buffer 粒度把通信与计算流水隐藏

多模态编码器优化（速览）

- **Dynamic CP**：大图沿 patch 维切到多设备、gather-KV 算注意力；每个 CP 组再分若干 sub-CP 组、多张大图**负载均衡**分配 → 通信占比不随规模增长
- **Bubble filling**：延续 K2.5 的 Decoupled Encoder Process，进一步分解 ViT 计算
 - 首批 micro-batch 的 ViT 前向同步先算，其余塞进**流水气泡**，反向同理
 - → ViT 计算几乎全被气泡吸收，**视觉编码器有效开销基本消除**

呼应总纲：多模态 3T 的三难题（不均 / 超显存 / ViT 关键路径）被 MoonEP / 激活管理 / bubble filling 分别接管。

Part 4 · 后训练 / RL

(简化)

RL 全景：三阶段 + 9 个专家

SFT (冷启动) → 按域×effort 做 RL 得 9 个专家 → MOPD 多教师蒸馏合并为单模型

- **RL 三大域** × {low, high, max} = **9 个专家** : general / general agents / coding agents
- scaling RL FLOPs → tool-call 步数与整体能力**一致提升**
- **算法：部分 rollout (partial rollout)**
 - 采样 N prompts × K completions ; λ 比例轨迹完成就**暂停生成**、让策略优化先走 (避免 straggler 长尾)
 - 长轨迹跨多迭代 → 数据 **staleness** → 靠**逐 token 正则**把更新限制在局部邻域, 容忍极端 off-policy
- **Reasoning-effort RL** : 每题给 token 预算, 超预算给 -1 ; 按 τ 从大到小课程得 max/high/low
- **Agentic GRM** : 锦标赛式二元比较 + rubric 强制协议 + verbosity 预算防 reward hacking

MOPD 蒸馏 + 部署感知后训练

MOPD (多教师 on-policy 蒸馏)

按域 d 与 effort e 用对应教师, 逐 token 稠密奖励:

$$r = \text{clip}\left(\text{sg log } \frac{\pi_{teacher}}{\pi_{student}}, -R_{max}, R_{max}\right)$$

- 稠密信号天然融入 RL 框架、支持 partial rollout

部署感知

- **Draft / EAGLE-3**: 把预训练 MTP 层微调成 draft; 输入融合第 1/4/末 AttnRes block 的低/中/高层特征
- **LK loss**: 直接优化投机解码**接受率**
— $-\log \sum \min(p, q)$ 而非 KL 代理
- draft 训练也在 QAT 配置下

RL 环境: 统一 white-box harness (防对单一 harness 过拟合) · 知识图谱引导任务合成 · kernel 优化 (含防 reward-hacking) · AET 自主执行 · personal assistant mock · web dev

推理 & Serving Infra (补充速览)

- **KDA-aware prefix cache** : MLA KV (随长度增长) 与 KDA 递归态 (固定大小) 生命周期完全不同, 但前缀**只有两者同边界一起恢复**才可复用
 - 统一分页块池 + 解耦"哈希粒度 vs 物理块粒度" → 可在 512-token 边界命中、深入大物理块内部
 - 并发一致性: 命中块跨组 pin、checkpoint 原子失效
- **专用 kernel** : KDA 解码 (只缓存投影输入、片上重建状态, 解决 MTP 拒绝无法回滚) · Block AttnRes 两阶段 · LatentMoE (下投影+router 融合、WarpDecode)
- **Fleet 级调度** : cache-aware affinity (一致性哈希 pin 主+备集群) · budget-based admission (按请求类分预算, 防长请求拖垮短请求 TTFT)

评测速览

- **整体**：紧随最强闭源（Claude Fable 5 / GPT-5.6 Sol），**稳定领先其他开源**
- **领先/SOTA**：ProgramBench · SWE-Marathon · BrowseComp · DeepSearchQA · MCPMark · OmniDocBench · **WebDev Arena（首个登顶的开源模型）**
- **短板**：research 级推理（HLE-Full / CritPt）、部分知识工作 Elo、hardened target 端到端 exploit
- **成本效率**：多个 coding/agentic 榜处于成本-效率前沿（BrowseComp 最佳分且约 GPT-5.6 Sol 一半成本）
- **Case study**：GPU kernel 优化 · 从零写编译器 MiniTriton · 48h 自主设计推理芯片 nano-kpu · 复现天体物理 I-Love-Q

演进脉络

K2 (全 MLA · 384 选 8 · Muon · SwiGLU · 128K)

→ **K2.5** (视觉 agentic · Decoupled Encoder)

→ **Kimi Linear** (提出 KDA · 3:1 KDA:MLA · 验证线性注意力可超全注意力)

→ **Attention Residuals** (深度维注意力 · 48B 验证)

→ **Kimi K3** (KDA + AttnRes + LatentMoE + Per-Head Muon + QAT + 全套 infra @ 2.8T / 1M)

每一代都在为下一代"验证一个新机制", K3 是集成大成。

收尾：五条方法论启示

1. **算法-系统协同是主线**：给 KDA 门"加下界" → 对角块上 Tensor Core；MoonEP "完美均衡" → 静态 shape + 零拷贝 + 免同步。**改算法时先想它对 kernel/通信/显存意味着什么。**
2. **把成熟维度的思想迁到新维度**：attention 从序列维迁到深度维（AttnRes）；CP 从 softmax 迁到线性注意力（KCP：转移矩阵分解 + 前缀扫描）。
3. **有界化 / 归一化 = 极端稀疏 + 低精度下的稳定器**：SiTU-GLU 有界 · LatentMoE 加 RMSNorm · KDA 门下界 · per-head 正交化。
4. **比较训练技巧要"各自调到最优再比"**（cosine vs WSD 的教训）。
5. **负载均衡从"启发式步长"升级为"有理论最优解的分位数"**（QB + 直方图全局估计）。

Q & A

谢谢！

备问：为何 3:1 而非全 KDA · NoPE 外推 · 896 选 16 如何训得动 · AttnRes 推理开销 · 与 DeepSeek 到底差在哪 —— 详见思路文档
DESIGN_NOTES.md

附录 · 预训练 Infra 深挖

(备用深讲页 · 详见 PRETRAIN_INFRA_DEEPDIVE.md)

附录 A1 · 并行组合的职责与耦合

PP(interleaved 1F1B + VPP) + EP + ZeRO-1 DP + Pipeline ZeRO-2 梯度分片 + CP(KCP)

维度	切什么	为什么不可或缺
PP(+VPP)	93 层按 stage	层多单层大；VPP 压 pipeline bubble
EP	896 专家按 rank	专家权重占大头，DP 复制会爆
ZeRO-1 DP	优化器状态分片	扩批量、通信最省
Pipeline ZeRO-2	梯度分片(+CPU)	在 PP 之上再压 GPU 峰值
CP(KCP)	序列维	1M 上下文放不下；KDA 递归态专门处理

- **关键约束**：EP 的 all-to-all、PP 的 P2P、DP 的 reduce **争抢同一带宽** → 全篇主线是 **用计算盖住每一条通信**
- **显存是全局零和**：ZeRO-2 把梯度下沉 CPU，是为给激活 offload / RL 的外部 KV pool 腾 HBM

附录 A2 · 1F1B + VPP : bubble 与激活峰值

- GPipe bubble 占比 $\approx (p - 1)/m$ (p =stage 数, m =micro-batch 数)
- **1F1B** : 不改 bubble 比例, 但稳态下每 stage 只驻留 $\sim p$ 份在飞激活 (而非全部 m 份) $\rightarrow$ **降激活峰值**
- **VPP** : 每物理 stage 再拆 v 个虚拟块 $\rightarrow$ bubble $\approx (p - 1)/(m \cdot v)$, 代价是**通信次数 $\times v$**
- **副作用 (后面要治)** : 暖机期各 stage 驻留激活**不均** $\rightarrow$ 附录 A7 的「PP 激活再平衡」

Fig.11 调度三条意图 :

1. shared expert 拆 SE1/SE2 派独立 stream $\rightarrow$ 填 all-to-all 空档
2. `reduce_grad` = reduce_scatter + onload + add + offload (= Pipeline ZeRO-2 + CPU 梯度)
3. EP-DR (dispatch 重算) 在反向 $\rightarrow$ memory-efficient MoE

附录 A3 · 常规 EP 的三连击（同源）

设 S =每 rank token 数, K =top-k, E =专家数, R =EP size。

1. **热 rank 决定迭代时间**：EP 是同步屏障，最慢 rank 拖住全部
2. **动态形状 → 碎片 → OOM**：每步每层路由计数都变，routed 激活 shape 变化 → 分配器反复申请释放 → 碎片累积
3. **每层 host ↔ device 同步**：grouped GEMM 需 `cu_seqlens`（设备上算的数据相关量）→ host 必须读回才能按形状发射 → 每层一次，CPU 卡在关键路径

三点**同源**：都是"token → 专家映射动态且不均"的后果。

治本 = 让映射结果对系统而言变成静态且均衡。

附录 A4 · MoonEP 机制：动态冗余专家

目标：每 rank 恰收 $S \times K$ 个 token（计算量完全相同）

手段：**冗余专家** —— 把热专家临时复制到别的 rank，长尾被摊平

- **前向**：从当前 router 输出**在线规划**冗余专家 → **预取**权重到本地槽 → 形状一致的 grouped GEMM
- **反向**：冗余专家梯度先 stage 到**本地 reduce buffer**，算完 **reduce** 回 **home rank**
- 权重布局 $[E+B, H, H']$ ： $[0, E)$ 本地专家、 $[E, E+B)$ 预取槽；训练 $B = E/R$
- **在线规划**：离线 ILP 求精确最优做**标尺**，线上用**近最优 GPU planning kernel**（开销可忽略、恒满足 E/R ）

附录 A5 · 为什么 buffer 是 $S \times K$ 而非 $S \times K \times R$ + 静态形状

Zero-copy 缓冲区

- 规划 kernel **预算每 token 目的地** → token 直接写到远端最终槽位，buffer 视图直接给计算（**无 comm → user 拷贝**）
- **DeepEP**：最坏某 rank 收下所有 R 个 rank 的 $S \times K$ → 需 $S \times K \times R$
- **MoonEP**：完美均衡恒收 $S \times K$ → **固定 $S \times K$** ，与倾斜无关

静态形状红利

- 形状静态已知 → **消除每层 MoE host 同步**、降 launch 开销
- **rank 内仍偏斜** → workload-aware GEMM 调度（分析代价模型 + 离线 autotune）
- shared 专家 GEMM 派独立 stream overlap

均衡 → 静态形状 → 免同步 + 零拷贝 + 无碎片，全是**免费红利**。

附录 A6 · MoonEP 对比总表

维度	DeepEP	ECHO / UltraEP	MoonEP
负载均衡	无（硬扛）	固定冗余/cap，可能无解	完美均衡，规划恒可行
零拷贝 buffer	最坏 $S \times K \times R$	—	固定 $S \times K$
计算形状	动态（每层 host 同步）	动态	静态（免同步）
高不均行为	通信恶化、碎片 → OOM	训练可能中断+手调	迭代时间持平、不 OOM

思想：用很小代价 ($\leq E/R$ 冗余专家 + 规划 kernel) 把"数据相关的不均"变成"系统层面的完全确定与均衡"。

附录 A7 · 显存账本：各手段省在哪

手段	省的是	代价
FP8 块量化激活	激活字节 ($\sim\frac{1}{2}$)	量化/反量化算力
offload / 远程 offload	HBM $\rightarrow$ CPU/远端 HBM	互联带宽 (被 overlap 盖住)
跨层重计算	激活保存	前向重算算力
Memory-efficient MoE	前向 output + dispatch 输入	少量逐元素 + 可 overlap 重算
Block AttnRes checkpoint	$O(Ld) \rightarrow O(Nd)$	AttnRes 重算
PP 激活再平衡	峰值 rank 的 HBM	跨 rank 传输(Mooncake)
Pipeline ZeRO-2 + CPU 梯度	GPU 梯度显存	CPU 内存 + offload
P2P Muon	全参 buffer + all-gather	P2P(流水隐藏)

统一激活管理器：把这些都做成 tensor 粒度、注解声明、可自由组合的"存储策略"；单内存池防多流碎片。

附录 A8 · P2P Muon : 定向通信 > 广播

- **约束** : Newton–Schulz 迭代 $X \leftarrow aX + bX(X^\top X) + \dots$ 是**整矩阵多项式** , 必须完整参数矩阵 $\rightarrow$ 无法在分片上独立做 ; 但优化器已按 DP 分片参数
- **朴素** : 全量 all-gather 完整参数 $\rightarrow$ **显存** (每 rank 多一份全参 buffer) + **通信** (all-gather 全参) 双爆炸
- **K3** : 每 rank 只 **P2P 拉回"自己负责更新的参数"的分片** (属主 $\rightarrow$ 更新者定向) $\rightarrow$ 重建自己要更新的矩阵即可
 - 临时 buffer 从 $O(P)$ 降到 $O(P/DP)$, **消除全参 buffer**
 - 按 model-chunk buffer 粒度把通信与计算**流水隐藏**

原理 : **只有"要更新某参数的 rank"才需要它的完整矩阵** $\rightarrow$ gather 应定向 , 而非广播。

附录 A9 · 组件耦合：谁解决谁 / 谁增强谁

- **痛点① EP 不均** → **MoonEP**：附带静态形状（免同步）、零拷贝（通信不膨胀）、无碎片（**反哺显存**）
- **痛点② 显存** → **组合拳**：统一激活管理 + memory-efficient MoE + Block AttnRes checkpoint + PP 激活再平衡 + Pipeline ZeRO-2 + P2P Muon → 腾出的 HBM 支撑更长序列 / RL 外部 KV pool
- **痛点③ ViT 方差** → **Dynamic CP + Bubble filling**（承 K2.5 DEP，思路同 Optimus）
- **贯穿**：每条通信（all-to-all / P2P / reduce / offload）都被某段计算盖住

两个"协同放大"：

1. **MoonEP 静态形状 × 单内存池** → 动态形状是碎片之源，消灭它后几乎不再碎片化：**均衡直接改善显存**
2. **KDA 下界化衰减（算法） × FlashKDA（系统）** → 加下界让对角块上 Tensor Core，FlashKDA 才能充分 overlap：**算法改动是系统优化的前提**